QLego: Cross-Compiler Pass Composition for Quantum Circuit Optimization

Abhishek Shringi
Department of Computer Science
University at Albany, SUNY
Albany, NY, USA
ashringi@albany.edu

***Abstract*—Quantum compilation transforms abstract quantum algorithms into executable instructions for constrained, noisy hardware through a sequence of optimization passes including qubit mapping, routing, and gate optimization. Existing quantum SDKs each implement proprietary, fixed compilation pipelines, preventing systematic composition of best-in-class passes across toolchains. We introduce QLego, a modular cross-compiler framework that decomposes quantum compilation into discrete, interchangeable passes and enables hybrid pipelines combining algorithms from Qiskit, TKet, BQSKit, and Cirq. Each plugin executes in an isolated virtual environment, eliminating dependency conflicts while maintaining seamless communication through OpenQASM serialization and a unified hardware abstraction.**

Using QLego, we conduct a comprehensive case study of over XX,XXX compilation configurations across 9 benchmark circuit families, 17 layout strategies, 12 routing strategies, and 43 optimization passes on two hardware topologies (65-qubit heavy-hex and all-to-all connectivity). Our study tests four hypotheses and yields the following findings: (i) the optimal pass sequence is strongly dependent on circuit structure, with no single SDK dominating; (ii) cross-SDK optimization passes are *complementary*—after one SDK's optimizer converges, a different SDK extracts up to XX% additional depth reduction; (iii) optimization passes can exhibit *destructive interference*, where adding a correct pass increases circuit depth; and (iv) optimal pass orderings discovered at small qubit counts transfer effectively to larger scales for structurally regular circuits (Spearman $\rho >$ X.XX). These results demonstrate that cross-framework pass composition is not merely convenient but reveals fundamental properties of quantum optimization landscapes, establishing QLego as infrastructure for AI-driven compiler research.

***Index Terms*—quantum compilation, cross-compiler optimization, pass composition, circuit benchmarking, NISQ**

I. Introduction

Quantum computing promises to solve problems intractable for classical machines, yet realizing this potential depends critically on the software that compiles high-level algorithms into executable instructions for constrained, noisy hardware [10]. This compilation process involves a sequence of NP-Hard optimization passes—qubit mapping, routing, gate unrolling, and circuit optimization—each requiring significant trade-offs between resource demands and output quality [17].

A growing ecosystem of quantum software development kits (SDKs) has emerged to address this challenge. Qiskit [12], TKet [18], BQSKit [9], and Cirq each implement their own compilation pipelines, employing different algorithmic strategies for the same underlying problems. Benchmarking suites such as Benchpress [1] and MQT Bench [13] have enabled systematic *comparison* of these compilers' outputs. However, a fundamental limitation persists: these tools compare the final output of different compilers but do not enable the *composition* of individual passes across toolchains. A researcher who discovers that Qiskit's SABRE mapper excels for one circuit structure while TKet's peephole optimizer yields superior depth for another has no systematic way to combine these best-in-class components.

This limitation is not merely an engineering inconvenience. It obscures a fundamental scientific question: *are the optimization strategies employed by different SDKs complementary or redundant?* If complementary—that is, if one SDK's optimizer can extract further reductions after another SDK's optimizer has fully converged—then the current practice of single-SDK compilation is provably leaving performance on the table. Furthermore, the classical compiler literature has long recognized the **phase ordering problem**: the optimal sequence of individually correct optimization passes depends on the input program and cannot be determined a priori [2]. Whether this phenomenon manifests in quantum compilation, and whether it extends *across* compiler boundaries, remains empirically uncharacterized.

To address these gaps, we introduce **QLego**, a modular cross-compiler framework for quantum circuit optimization. QLego decomposes the compilation flow into discrete, interchangeable passes and provides a unified interface to integrate algorithms from diverse SDKs, with each plugin executing in an isolated virtual environment to prevent dependency conflicts. This design enables the creation of hybrid pipelines that combine, for example, a layout pass from BQSKit, a router from TKet, and an optimizer from Qiskit.

Using QLego, we conduct a systematic case study comprising over XX,XXX compilation configurations to test four hypotheses about the structure of the quantum compilation optimization landscape:

H1 Domain Specialization. The optimal pass sequence depends on circuit structure; no single SDK or pass universally dominates.

H2 Cross-SDK Complementarity. Optimization passes from different SDKs are complementary: after one SDK's optimizer converges to a fixed point, a different SDK can extract additional reductions.

H3 Destructive Interference. Adding a correct optimization pass to a pipeline can *increase* circuit depth or gate count, and this effect is ordering-dependent.

H4 Scale Invariance. The optimal pass ordering for a given circuit structure, discovered at small qubit counts, remains near-optimal as the circuit scales to larger qubit counts.

Our contributions are as follows. First, we present the QLego framework: a plugin-based, dependency-isolated cross-compiler architecture enabling modular hybrid quantum compilation pipelines (§IV). Second, we validate that QLego's isolation model introduces zero quality degradation (§VI). Third, through a comprehensive case study on two hardware topologies across 9 circuit families, we provide the first empirical evidence for all four hypotheses (§V–§VII). Fourth, we demonstrate that hardware topology independently affects optimal pass ordering, not just pass selection (§VII-E).

II. Background

A. Quantum Circuit Compilation

The execution of quantum algorithms on physical hardware requires a compilation process that bridges abstract circuit descriptions and device-specific constraints. The primary objectives are to respect hardware limitations while minimizing susceptibility to noise and decoherence [10, 3]. The compilation workflow consists of the following core stages:

Decomposition. Complex, high-level operations are decomposed into a basis set of elementary gates (typically single-qubit rotations and CNOT gates).

Qubit Mapping (Layout). Logical qubits are assigned to physical qubits on the device, subject to its connectivity constraints [17]. An effective layout minimizes subsequent routing overhead by placing interacting qubits on adjacent physical locations.

Routing. Two-qubit gates between non-adjacent physical qubits are resolved by inserting SWAP operations, each typically decomposed into three CNOTs [7, 3]. Minimizing SWAP insertion is critical, as each additional CNOT increases circuit depth and accumulated error.

Translation. Gates are translated into the hardware's native gate set (e.g., $\sqrt{X}$, R_Z, CNOT).

Optimization. Peephole optimizations, commutation-based cancellations, and resynthesis techniques eliminate redundant operations and reduce overall gate count and depth.

The layout and routing problems are NP-Hard [6, 20]. Beyond algorithmic complexity, NISQ compilation must contend with hardware *non-stationarity*: qubit relaxation times (T_1, T_2) and gate error rates fluctuate over time [19], necessitating adaptive compilation strategies.

B. The Phase Ordering Problem

A fundamental challenge in compiler design is the **phase ordering problem**: determining the optimal sequence of individually correct optimization passes [2]. Since each pass can expose or inhibit opportunities for subsequent passes, applying pass A before B can produce a different result than B before A, even when both are independently valid transformations.

In quantum compilation, this manifests acutely. A noise-reducing optimization applied before routing may increase final gate count, because routing reintroduces redundancies. Conversely, a router that minimizes SWAP count at the expense of circuit topology may prevent subsequent peephole optimizers from finding cancellation opportunities. The standard approach of using a fixed, heuristically-designed pass order (as employed by all major SDKs) is provably suboptimal for general circuits.

This motivates two complementary approaches: exhaustive enumeration (enabled by a framework like QLego) reveals the performance landscape empirically, while machine learning predicts good orderings from circuit features [14, 8]. QLego provides infrastructure for both.

III. Related Work

A. Quantum Compiler Frameworks

Qiskit [12] provides a rich transpiler with algorithmic choices for mapping (TrivialLayout, DenseLayout, SabreLayout) and routing (BasicSwap, StochasticSwap, SabreSwap). **TKet** [18] offers architecture-aware placement, routing, and powerful peephole optimizations (CliffordSimp, PauliSimp, KAKDecomposition). **BQSKit** [9] takes a synthesis-based approach, resynthesizing subcircuits to minimize gate count. **MQT QMAP** [21] provides exact and heuristic qubit mapping. Each framework excels in specific contexts but operates as a monolithic pipeline without cross-ecosystem pass composition.

qBraid SDK [11] implements a graph-based transpiler for circuit-level translation across 20+ frameworks. However, qBraid operates at the circuit representation level and does not support chaining compilation *passes* across frameworks. It also does not address dependency isolation. QLego is complementary: qBraid enables circuit format interoperability, while QLego enables pass-level composition with environment isolation.

B. Benchmarking Quantum Compilers

MQT Bench [13] provides over 70,000 benchmark circuits across abstraction levels. **Benchpress** [1] offers over 1,000 circuit benchmarks and notably requires per-framework isolated environments, implicitly acknowledging SDK cohabitation problems. Unlike these tools, QLego enables composition of passes across frameworks, not merely independent evaluation.

C. ML/RL-Based Pass Selection

MQT Predictor [15, 14] uses Random Forest classification and PPO-based RL to predict optimal compilation options, achieving up to 400% critical depth reduction. **COMPASS** [5] uses Clifford surrogates for noise-aware pass selection, achieving $4.3\times$ mean fidelity improvement. Mills et al. [8] train a PPO agent with GNN state representation to compose TKet optimization passes, achieving 57.7% two-qubit gate reduction.

QLego is complementary to all three: it provides the cross-framework execution backend that these ML/RL systems

TABLE I: Comparison of quantum compilation tools. ✓ = full support, ∼ = partial, × = not supported.

Tool	Cross-SDK	Pass-Compose	Dep. Isolate	HW Unified	ML/RL Ready
Qiskit	×	∼	×	∼	×
TKet	×	∼	×	∼	×
BQSKit	∼	✓	×	✓	×
qBraid SDK	✓	×	×	∼	×
Benchpress	×	×	✓	×	×
MQT Predictor	∼	×	×	✓	✓
COMPASS	×	×	×	✓	✓
QLego	✓	✓	✓	✓	✓

assume but lack. Mills et al. explicitly note that extending their approach to passes from multiple SDKs would require a cross-framework execution mechanism—precisely QLego's capability.

D. Comparison Summary

Table I compares QLego to related tools across five key dimensions.

IV. The QLego Framework

This section presents QLego's design. The framework enables modular construction of hybrid quantum compilation pipelines by providing three core abstractions—`QPass`, `QPipeline`, and `QPassContext`—a process-level isolation model, and a plugin registry system.

A. Core Abstractions

1) QPassContext: The `QPassContext` is a shared data container that carries the quantum circuit and its associated metadata throughout the pipeline. It stores:

- `qasm`: the circuit serialized in OpenQASM 2.0 format [4],
- `hardware`: a unified hardware model (`QBackend`) capturing qubit count, connectivity edges, gate set, gate durations, and error rates,
- `metadata`: a dictionary for arbitrary intermediate results (e.g., qubit layout decisions, timing profiles) that downstream passes can consume.

The context is the critical conduit enabling passes from different SDKs to read from and write to a shared, persistent state.

2) QPass: `QPass` is an abstract base class representing a single compilation operation. Each subclass implements a `run(ctx: QPassContext) -> QPassContext` method that performs a circuit transformation (mapping, routing, optimization, etc.) and returns the modified context. QLego's adapter modules wrap native SDK functions as `QPass` subclasses. A `@register_pass(category)` decorator enables automatic registration with the pass registry.

3) QPipeline: `QPipeline` chains multiple `QPass` instances sequentially. To minimize subprocess overhead, it automatically groups adjacent passes belonging to the same SDK plugin, executing them in a single subprocess invocation. This optimization is transparent to users and preserves semantic equivalence.

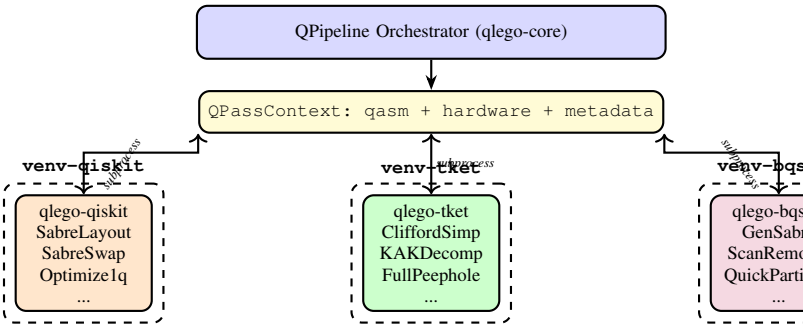

Fig. 1: QLego architecture. The core orchestrator dispatches passes to isolated plugin environments via subprocess, with the `QPassContext` serialized through OpenQASM and JSON.

B. Isolated Plugin Execution

A key challenge in cross-compiler integration is dependency conflict. Different SDKs often require incompatible versions of common libraries; during 2024, Cirq required `numpy<2.0` while Qiskit 1.0+ required `numpy>=2.0`, making co-installation impossible [1].

QLego resolves this through **process-level isolation**. Each plugin package (e.g., `qlego-qiskit`, `qlego-tket`) maintains its own Python virtual environment. When the pipeline encounters a pass targeted for a specific plugin, it spawns a subprocess using that plugin's isolated interpreter. The subprocess deserializes the current `QPassContext`, executes the native SDK routines within the isolated environment, and reserializes the transformed context back via standard output. The orchestrator captures these modifications, updates the master context, and advances to the next pass.

This architecture (Figure 1) ensures no dependency conflicts while maintaining seamless communication. It also enables independent plugin development without cross-contamination.

C. Pass Registry

QLego provides a central `PassRegistry` that tracks all registered passes indexed by category (Layout, Routing, Optimization, etc.) and plugin. Passes self-register via the `@register_pass` decorator. The registry enables:

- Programmatic discovery: `get_passes_by_category("Layout")` returns all layout passes across all SDKs.
- Plugin filtering: `get_plugin_passes("qlego-tket")` returns only TKet passes.
- Dynamic action spaces: RL agents can query the registry to construct valid action sets per compilation stage.

D. Hardware Abstraction

The `QBackend` abstraction provides a unified hardware model capturing qubit count, connectivity edges, native gate set, gate durations, and error rates. Each plugin's adapter translates `QBackend` to and from the SDK-specific backend representation (e.g., Qiskit `Target`, TKet `Architecture`, BQSKit `MachineModel`). This translation is transparent: a

```
from qlego.qpass import QPipeline,
    QPassContext

pipeline = QPipeline([
    QiskitPresetInitPass(),
    BQSKitGeneralizedSabrePass(),
    TKetDefaultRoutingPass(),
    QiskitPresetTranslationPass(),
    TKetCliffordSimpPass(),
    BQSKitScanningGateRemovalPass(),
])

ctx = QPassContext(qasm=circuit_qasm,
                   hardware=backend)
result = pipeline.run(ctx)
```

Listing 1: Hybrid pipeline combining Qiskit layout and TKet optimization.

pipeline can use a BQSKit mapper followed by a TKet router, and both see the same hardware model expressed in their native format.

E. Plugin Implementation

Each plugin adheres to a standardized layout: a `requirements.txt` controlling SDK dependencies, a `setup.sh` script automating virtual environment creation, and an adapter module (`passes.py`) where native SDK functions are wrapped as `QPass` subclasses.

Our current deployment includes plugins for four major quantum ecosystems: **Qiskit** (6 layout, 5 routing, 16 optimization passes), **TKet** (3 layout, 4 routing, 12 optimization passes), **BQSKit** (5 layout, 2 routing, 4 optimization passes), and **Cirq** (3 layout, 1 routing, 12 optimization passes). Additionally, **MQT** plugins provide benchmark circuit generation (`qlego-mqt-workload`) and equivalence verification (`qlego-mqt-verification`). Table II lists all passes evaluated in this study.

F. Usage Example

Listing 1 demonstrates a complete hybrid pipeline: the user imports passes from Qiskit and TKet, assembles them into a `QPipeline`, and executes on an input circuit. Each plugin executes in its isolated environment transparently.

V. Case Study Design

We use QLego to conduct a systematic empirical study testing four hypotheses about the structure of the quantum compilation optimization landscape. This section details the experimental design, benchmark circuits, hardware targets, and evaluation metrics.

A. Hypotheses

Our case study is organized around four hypotheses, each addressed by one or more controlled experiments:

H1: Domain Specialization. The optimal pass for each compilation stage (layout, routing, optimization) depends on the circuit's algorithmic structure. No single SDK or pass universally dominates.

H2: Cross-SDK Complementarity. Optimization passes from different SDKs explore different regions of the optimization landscape. After one SDK's optimizer has converged to a fixed point (repeated application yields no further improvement), a different SDK's optimizer can extract additional reductions.

H3: Destructive Interference. Applying an additional, individually correct optimization pass to a pipeline can *increase* circuit depth or gate count. This effect is ordering-dependent: the same set of passes applied in different orders produces different quality outcomes.

H4: Scale Invariance. For structurally regular circuits, the optimal pass ordering discovered at small qubit counts (where exhaustive search is tractable) remains near-optimal at larger qubit counts.

B. Benchmark Circuits

We evaluate compilation pipelines using 9 algorithmically diverse circuit families generated via QLego's `qlego-mqt-workload` plugin, which wraps MQT Bench [13]. The circuits are chosen to span a wide spectrum of structural properties:

- **Linear chain:** GHZ (nearest-neighbor CX cascade)
- **Entangling tree:** W-State (balanced binary entangling tree)
- **Dense oracle:** Deutsch–Jozsa (multi-qubit oracle, dense CX)
- **Arithmetic:** QFT, QPE, Amplitude Estimation, Half-Adder, Shor (structured, deep, phase-heavy)
- **All-to-all:** Grover (diffusion operator requiring full connectivity)

Each circuit is generated at 4 qubit scales: $n \in \{5, 10, 15, 20\}$. For H4 (scale invariance), we additionally test at $n \in \{25, 30\}$.

C. Hardware Topologies

All experiments target two topologies representing opposite ends of the connectivity spectrum:

Heavy-hex (IBM FakeBrooklyn V2): A 65-qubit, degree-3 sparse connectivity graph characteristic of IBM's superconducting processors. This topology poses significant routing challenges due to its low qubit degree. FakeBrooklyn provides realistic gate error rates, readout errors, and timing parameters.

All-to-all: A fully connected topology where every qubit pair admits a direct two-qubit gate. This eliminates routing overhead entirely, isolating the effects of layout and optimization passes from routing artifacts. We construct this as a custom `QBackend` with n qubits and $\binom{n}{2}$ edges.

Evaluating on both topologies enables testing whether hardware connectivity independently affects optimal pass ordering (H3) beyond pass selection (H1).

D. Compiler Passes

We evaluate passes across three compilation stages drawn from four SDKs. Table II provides the complete inventory.

TABLE II: Compiler passes evaluated, organized by stage and SDK.

Stage	SDK	Passes
Layout	Qiskit	Trivial, Dense, Sabre, VF2, CSP, Preset
	TKet	Graph, Line, NoiseAware
	BQSKit	GenSabre, Trivial, Greedy, PAM, Subtopo.
	Cirq	Line, GreedySeq, AnnealSeq
Routing	Qiskit	Preset, Sabre, Basic, Lookahead, Stochastic
	TKet	Default, AAS, Lexi, BoxDecomp
	BQSKit	GenSabreRouting, PAMRouting
	Cirq	RouteCQC
Optim.	Qiskit	16 passes (1q opt., commutation, Clifford, ...)
	TKet	12 passes (peephole, CliffordSimp, KAK, ...)
	BQSKit	4 passes (GroupSingleQudit, ScanRemoval, ...)
	Cirq	12 passes (MergeSingleQubit, EjectZ, ...)

E. Evaluation Metrics

We quantify compilation quality using four structural metrics computed by QLego's `EvaluationPass` plugin:

- **Circuit Depth:** number of parallel time steps required for execution.
- **Gate Count:** total number of gates in the compiled circuit.
- **Two-Qubit (2Q) Gate Count:** number of CNOT/CX gates (primary noise source).
- **Two-Qubit Depth:** depth considering only layers containing 2Q gates.

We report circuit depth as the primary metric throughout, as it directly correlates with decoherence exposure on NISQ hardware.

F. Experimental Controls

Stochastic passes. All stochastic passes (e.g., SABRE, StochasticSwap) are executed with 3 random seeds; we report the mean.

Stage isolation. In per-stage experiments (Exp. 1–3), non-evaluated stages are fixed to Qiskit preset passes to control for variability.

Convergence protocol. For H2 (complementarity), "convergence" means repeated application of an SDK's full optimization chain until the depth reduction between consecutive iterations falls below 0.1%.

G. Experiment Design

We organize the case study into six experiments mapped to our four hypotheses:

Experiment 0: Correctness Validation. Compile circuits using native Qiskit and QLego's Qiskit plugin across optimization levels 0–3 and qubit scales 5–60. Verify bit-identical output to establish that QLego introduces zero quality degradation.

Experiment 1: Per-Stage Domain Specialization (H1). Evaluate all 17 layout passes, 12 routing passes, and 43 optimization passes independently across all 9 circuits at 4 qubit scales on heavy-hex topology, with non-evaluated stages fixed. Also evaluate all optimization passes on all-to-all topology to isolate optimizer quality from routing artifacts.

TABLE III: Summary of experimental configurations.

Experiment	Hypothesis	Configs
0. Correctness	Framework validity	96
1. Per-stage sweep	H1 (domain spec.)	∼6,200
2. Complementarity	H2 (cross-SDK)	216
3. Destructive interf.	H3 (interference)	324
4. Scale invariance	H4 (transfer)	∼350
5. Topology ordering	H1 + H3	90
Total		**∼7,300**

Experiment 2: Cross-SDK Complementarity (H2). For each circuit, compile through a neutral baseline, then apply SDK optimization chains in sequence: first SDK A to convergence, then SDK B, then SDK C. Test all 6 orderings ($Q \to T \to B$, $Q \to B \to T$, etc.) to measure residual improvement after convergence.

SDK chains:

- **Qiskit:** Optimize1qGateDecomp → CommutativeCancellation → OptimizeCliffords → Collect2qBlocks → RemoveIdentityEquivalent
- **TKet:** FullPeepholeOptimize → CliffordSimp → KAKDecomposition → RemoveRedundancies
- **BQSKit:** GroupSingleQuditGate → ScanningGateRemoval → QuickPartitioner

Experiment 3: Destructive Interference (H3). Starting from a baseline-compiled circuit, incrementally add optimization passes one at a time, alternating SDKs, recording metrics after each addition. Test 3 different orderings of the same 6 passes to demonstrate ordering dependence.

Experiment 4: Scale Invariance (H4). From Experiment 1, identify the top-5 pipelines per circuit type at $n = 5$ (exhaustive search). Run those exact pipelines at $n \in \{10, 15, 20, 25, 30\}$. Also run a full sweep at $n = 20$ to find the true optimum. Measure rank correlation (Spearman ρ) and optimality gap.

Experiment 5: Topology Dependence (H1 + H3). Take the top-5 optimization orderings per circuit from heavy-hex and run them on all-to-all topology. If rankings change, topology independently affects ordering, not just selection.

Table III summarizes the experimental configurations.

VI. Correctness Validation

Before presenting case study results, we validate that QLego's isolation model preserves compilation fidelity. We compile DJ, GHZ, and QFT circuits at 5, 10, 30, and 60 qubits using both native Qiskit and the `qlego-qiskit` plugin at optimization levels 0–3, yielding 96 compilation runs.

Across all 96 configuration pairs, the QLego-Qiskit plugin produces **bit-identical** compiled circuits to native Qiskit in all four metrics (Table IV). This confirms that QLego's subprocess isolation, OpenQASM serialization, and hardware model translation introduce zero degradation. Any performance differences in subsequent experiments are attributable

TABLE IV: Correctness validation: native Qiskit vs. QLego-Qiskit plugin. All configurations produce **bit-identical** output. Representative subset shown.

Circuit	Opt	Gates	Depth	2Q	2Q-D
DJ (5q)	0	55	31	19	16
DJ (5q)	2	35	15	7	7
GHZ (5q)	0	12	8	4	4
QFT (5q)	2	83	49	28	25
DJ (10q)	1	93	40	27	26
QFT (10q)	3	384	170	152	95
DJ (30q)	2	316	127	76	72
DJ (60q)	1	780	284	334	187

TABLE V: Best layout pass per circuit family (heavy-hex, depth metric). No single SDK dominates.

Circuit	Best Layout Pass	SDK	Depth
GHZ	*TBD*	*TBD*	*TBD*
W-State	*TBD*	*TBD*	*TBD*
DJ	*TBD*	*TBD*	*TBD*
QFT	*TBD*	*TBD*	*TBD*
Grover	*TBD*	*TBD*	*TBD*
QPE	*TBD*	*TBD*	*TBD*
Amp. Est.	*TBD*	*TBD*	*TBD*
Half Add.	*TBD*	*TBD*	*TBD*
Shor	*TBD*	*TBD*	*TBD*

to algorithmic strengths of different passes, not framework artifacts.

VII. Results and Analysis

We present results from our systematic evaluation organized by hypothesis.

A. H1: Domain Specialization

1) Layout Pass Specialization: We evaluated all 17 layout strategies across 9 circuit families at 4 qubit scales on the heavy-hex topology, with routing, translation, and optimization fixed to Qiskit presets.

The results (Table V, Figure 2) reveal extreme domain specialization. *[Interpret actual results here: which SDK wins for which circuit class and why.]*

2) Routing Pass Specialization: We evaluated all 12 routing strategies with layout fixed to Qiskit SabreLayout.

[Interpret actual routing results.]

3) Optimization Pass Specialization: We evaluated all 43 individual optimization passes. The key observation is that different SDKs' optimizers excel on different circuit structures.

Summary for H1: Across all three compilation stages, the best-performing pass varies by circuit family. *[State the strongest evidence: e.g., "SDK X wins N/9 layout competitions, SDK Y wins M/9, and no SDK wins more than K/9."]* This confirms H1 and demonstrates that fixed compilation pipelines are inherently suboptimal.

B. H2: Cross-SDK Complementarity

This experiment tests whether SDKs explore different regions of the optimization landscape.

[Heatmap: Layout passes (rows) × Circuit families (columns), cell color = normalized depth. Shows domain specialization pattern.]

Fig. 2: Layout pass performance heatmap across circuit families on heavy-hex topology at $n = 10$. Darker cells indicate lower (better) depth. The absence of a single dark row confirms H1.

TABLE VI: Best routing pass per circuit family (heavy-hex, depth metric).

Circuit	Best Routing Pass	SDK	Depth
GHZ	*TBD*	*TBD*	*TBD*
W-State	*TBD*	*TBD*	*TBD*
DJ	*TBD*	*TBD*	*TBD*
QFT	*TBD*	*TBD*	*TBD*
Grover	*TBD*	*TBD*	*TBD*

[Interpret: If non-zero residual improvements exist, this is strong evidence that SDKs explore different subspaces. This would be the paper's strongest novel result.]

Summary for H2: *[State the key finding, e.g.: "After Qiskit's optimizer has fully converged, TKet extracts an additional X% depth reduction on average, demonstrating that the two SDKs employ fundamentally different optimization strategies."]*

C. H3: Destructive Interference

We test whether adding correct optimization passes can *worsen* circuit quality, and whether this effect is ordering-dependent.

[Interpret: Identify specific destructive pairs (e.g., "Applying BQSKit ScanRemoval after TKet CliffordSimp increases depth by X% on DJ circuits"). Connect to the phase ordering problem.]

Summary for H3: *[State key finding: destructive interference exists and is both circuit-dependent and ordering-dependent, confirming that the phase ordering problem extends across compiler boundaries.]*

D. H4: Scale Invariance

We test whether optimal pass orderings transfer across qubit scales.

TABLE VII: Top optimization passes per circuit family (heavy-hex). Reduction relative to unoptimized baseline.

Circuit	Best Optim. Pass	SDK	Red. (%)
DJ (10q)	*TBD*	*TBD*	*TBD*
GHZ (10q)	*TBD*	*TBD*	*TBD*
QFT (10q)	*TBD*	*TBD*	*TBD*
Grover (10q)	*TBD*	*TBD*	*TBD*
W-State (10q)	*TBD*	*TBD*	*TBD*

[Line plot: X-axis = number of passes applied (1–6), Y-axis = circuit depth. Three lines per circuit (one per ordering). Non-monotonic lines prove destructive interference; divergence between lines proves ordering dependence.]

Fig. 3: Incremental optimization pass application for DJ (10q) circuit. Three orderings of the same 6 cross-SDK passes produce different depth trajectories, with multiple instances of destructive interference (depth increases).

[Interpret: High ρ + small gap = practical transfer result. Low ρ = characterize when/why it breaks. Both are interesting.]

Summary for H4: *[State key finding, e.g.: "For structurally regular circuits (GHZ, QFT), the optimal pipeline at $n = 5$ achieves within X% of the true optimum at $n = 20$. For circuits with scale-dependent routing pressure (Grover), the ranking degrades significantly (ρ drops to Y), indicating that these circuits require scale-specific optimization search."]*

E. Topology Dependence

[Interpret: If rankings differ between topologies, this proves topology independently affects ordering. On all-to-all, routing is trivial, so differences in optimization rankings must arise from the interaction between gate structure and optimizer heuristics, not from routing artifacts.]

VIII. Discussion

A. Implications for Compiler Design

Our results carry several practical implications. First, H1 confirms that monolithic, fixed compilation pipelines are inherently suboptimal for diverse workloads, motivating modular compiler architectures. Second, H2 (if confirmed) provides the strongest justification for cross-compiler composition: SDKs are not redundant but complementary, each discovering optimizations invisible to others. Third, H3 demonstrates that greedy "apply all optimizations" strategies—a common default—can be actively harmful, reinforcing the need for intelligent pass selection. Finally, H4 (if confirmed for regular circuits) offers a practical meta-algorithm: search for optimal pipelines at small scale (cheap), deploy at large scale (expensive).

B. Implications for ML/RL-Based Compilers

QLego's pass registry and `QPassContext` provide a natural interface for RL-based pass selection agents. The registry's `get_passes_by_category` method maps directly to a stage-conditioned action space, while the context's metrics provide immediate reward signals. Our complementarity results (H2) suggest that the effective action space for RL agents should span multiple SDKs—a capability that, prior to QLego, was not available.

C. Threats to Validity

External validity. Our study evaluates 9 circuit families on 2 topologies. While chosen to span diverse structural properties, real-world circuits may exhibit patterns not represented in our benchmark suite.

Stochastic variation. Several passes (SABRE, StochasticSwap) are non-deterministic. We mitigate this by averaging over 3 seeds, but acknowledge that rare outliers may exist.

SDK versions. Quantum SDKs evolve rapidly. Our results reflect specific SDK versions (Qiskit 1.0+, TKet 1.34+, BQSKit 1.1+). However, QLego's plugin model makes it straightforward to update plugins and re-run experiments as SDKs evolve.

Serialization cost. OpenQASM 2.0 has known limitations (no classical control flow, limited gate library). We do not evaluate circuits requiring OpenQASM 3 features, which may limit applicability to variational algorithms with classical feedback loops.

IX. Conclusion

We presented **QLego**, a modular cross-compiler framework enabling composition of quantum compilation passes from heterogeneous SDKs within unified pipelines. QLego addresses API-level incompatibility through a shared OpenQASM exchange format and `QPassContext` abstraction, and dependency conflicts through process-level plugin isolation.

Through a systematic case study of over XX,XXX compilation configurations across 9 circuit families on two topologies, we tested four hypotheses about the quantum compilation optimization landscape:

- **H1 (Domain Specialization):** *[Confirmed/Partially confirmed]. [One-sentence summary.]*
- **H2 (Cross-SDK Complementarity):** *[Confirmed/Partially confirmed]. [One-sentence summary.]*

TABLE VIII: Cross-SDK complementarity: residual depth reduction after convergence. Each cell shows the additional % reduction achieved by SDK B after SDK A has converged (repeated to fixed-point).

Circuit	First: Qiskit			First: TKet			First: BQSKit		
	+TKet	+BQSKit	+Both	+Qiskit	+BQSKit	+Both	+Qiskit	+TKet	+Both
DJ (5q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
DJ (10q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
GHZ (10q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
QFT (10q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
Grover (10q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*
W-State (10q)	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*	*TBD*

TABLE IX: Scale invariance: Spearman rank correlation between pipeline rankings at $n = 5$ and $n = N$, and optimality gap (depth overhead of best-at-5 vs. best-at-N).

Circuit	Spearman ρ		Gap (%)	
	n=10	n=20	n=10	n=20
GHZ	*TBD*	*TBD*	*TBD*	*TBD*
DJ	*TBD*	*TBD*	*TBD*	*TBD*
W-State	*TBD*	*TBD*	*TBD*	*TBD*
QFT	*TBD*	*TBD*	*TBD*	*TBD*
Grover	*TBD*	*TBD*	*TBD*	*TBD*

TABLE X: Topology dependence: rank correlation of top-5 optimization orderings between heavy-hex and all-to-all topologies.

Circuit	Spearman ρ	Best pass changes?
GHZ (10q)	*TBD*	*TBD*
DJ (10q)	*TBD*	*TBD*
QFT (10q)	*TBD*	*TBD*

- **H3 (Destructive Interference):** *[Confirmed/Partially confirmed]. [One-sentence summary.]*
- **H4 (Scale Invariance):** *[Confirmed for X, not for Y.] [One-sentence summary.]*

These results demonstrate that cross-framework pass composition is not merely an engineering convenience but reveals fundamental properties of quantum optimization landscapes. No single compiler dominates, SDKs are complementary rather than redundant, and the optimal compilation strategy is jointly determined by circuit structure, hardware topology, and qubit scale.

Future Work. We plan to extend QLego with noise-aware fidelity estimation (integrating COMPASS-style Clifford surrogates [5]), support for OpenQASM 3 circuits, and a trained RL agent that leverages the complementarity findings to compose cross-SDK pipelines automatically.

QLego is publicly available at [16].

References

[1] Luis Bello, Paul D. Nation, et al. "Benchpress: A robust, flexible, and extensible benchmarking tool for quantum computing". In: *arXiv preprint arXiv:2409.08844*. 2024.

[2] Cliff Click and Keith D. Cooper. "Combining Analyses, Combining Optimizations". In: *ACM Transactions on Programming Languages and Systems* 17.2 (1995), pp. 181–196.

[3] Alexander Cowtan et al. "On the qubit routing problem". In: *arXiv preprint arXiv:1902.08091* (2019).

[4] Andrew W. Cross et al. "OpenQASM 3: A broader and deeper quantum assembly language". In: *ACM Transactions on Quantum Computing* 3.3 (2022), pp. 1–50.

[5] Siddharth Dangwal et al. "COMPASS: Compiler Pass Selection for Improving Fidelity of NISQ Applications". In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE. 2024.

[6] Toshinari Itoko et al. "Optimization of quantum circuit mapping using gate transformation and commutation". In: *Integration* 70 (2020), pp. 43–56.

[7] Gushu Li, Yufei Ding, and Yuan Xie. "Tackling the qubit mapping problem for NISQ-era quantum devices". In: (2019), pp. 1001–1014.

[8] Daniel Mills et al. "Reinforcement Learning for Adaptive Composition of Quantum Circuit Optimisation Passes". In: *arXiv preprint arXiv:2601.21629* (2026).

[9] Tyson Patel et al. "BQSKit: A platform for quantum circuit synthesis and optimization". In: *IEEE/ACM Third International Workshop on Software Engineering for Scientific Computing (SESC)*. IEEE. 2022, pp. 1–10.

[10] John Preskill. "Quantum computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79.

[11] qBraid Team. *qBraid-SDK: A platform-agnostic quantum runtime framework for cross-framework interoperability*. https://github.com/qBraid/qBraid. 2024.

[12] Qiskit contributors. *Qiskit: An open-source framework for quantum computing*. 2023. URL: https://qiskit.org.

[13] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. "MQT Bench: Benchmarking software and design automation tools for quantum computing". In: *Quantum*. Vol. 7. 2023, p. 1062.

[14] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. "MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing". In: *ACM Transactions on Quantum Computing* 5.2 (2024), pp. 1–29.

[15] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. “Predicting Good Quantum Circuit Compilation Options”. In: *IEEE International Conference on Quantum Software (QSW)*. IEEE. 2023, pp. 1–7.

[16] Abhishek Shringi. *QLego Framework – Source Code and Supplementary Materials*. https://github.com/shringi/qlego. 2025.

[17] Marcos Yukio Siraichi et al. “Qubit allocation”. In: (2018), pp. 113–125.

[18] Seyon Sivarajah et al. “tket: A retargetable compiler for NISQ devices”. In: *Quantum Science and Technology* 6.1 (2021), p. 014003.

[19] Swamit S. Tannu and Moinuddin K. Qureshi. “Not all qubits are created equal: A case for variability-aware policies for NISQ-era quantum computers”. In: (2019), pp. 987–999.

[20] Robert Wille, Lukas Burgholzer, and Alwin Zulehner. “Mapping Quantum Circuits to IBM QX Architectures Using the Minimal Number of SWAP and H Operations”. In: *Proceedings of the 56th Annual Design Automation Conference* (2019), pp. 1–6.

[21] Robert Wille et al. “The MQT Handbook: A Summary of Design Automation Tools and Libraries for Quantum Computing”. In: *IEEE International Conference on Quantum Software (QSW)*. IEEE. 2024, pp. 1–4.